

Matrix Context: Mixture-of-Contexts RAG for Robust and Inspectable Agent Memory

Routing typed context experts improves robustness under paraphrased and adversarial retrieval conditions

Ruslan Magana Vsevolodovna
Independent Researcher, Genova, Italy
contact@ruslanmv.com ruslanmv.com

June 2026

Abstract

We introduce *Mixture-of-Contexts RAG* (MoC-RAG), a routed retrieval architecture for agent memory that selects a small set of *typed context experts* *before* retrieving, then assembles a token-budgeted, inspectable context pack. To test the architecture beyond a keyword-aligned fixture, we build a public benchmark of typed agent memory (1,000 context items, 600 queries, eight experts, six domains) in which every gold fact is surrounded by five kinds of *hard negative* and queried in five styles ranging from direct to adversarial, with the same test topics phrased into parallel **keyword**, **paraphrased**, and **adversarial** splits. On this benchmark MoC-RAG improves robustness over flat RAG baselines: under adversarial lexical shift a strong lexical baseline (BM25) drops from 100% to 64% Recall@8 (−36 points), whereas MoC-RAG holds within ∼17 points and overtakes BM25 on the adversarial split by ∼15 points, while carrying roughly half the hard distractors of dense, hybrid, metadata-filtered, and reranked RAG at 95–100% expert-routing accuracy. We are careful not to overclaim: BM25 remains strong on keyword-aligned queries, and MoC-RAG’s advantage is robustness and context efficiency under typed, distractor-heavy, paraphrased and adversarial conditions—the regime real agent memory inhabits. All figures and tables in this paper are regenerated from committed result artifacts by the accompanying scripts.

1 Introduction

Retrieval-augmented generation [3] has largely settled into one pattern: embed everything into a single flat index and retrieve the nearest chunks for every query. For agent memory—where the store mixes user preferences, project decisions, code, policies, episodes, and documents—this is both wasteful and opaque. It spends a fixed prompt budget without regard to the *type* of what it retrieves, and it offers no account of *why* a given item was chosen.

Matrix Context proposes a different shape, which we call *Mixture-of-Contexts RAG* (MoC-RAG). Rather than retrieving from one undifferentiated store, it routes each query to the smallest useful subset of typed *context experts*, retrieves with a hybrid lexical and dense fusion inside them, and assembles a token-budgeted pack scored by relevance, importance, recency, and a redundancy penalty—with every selection explainable through an **inspect** interface.

The central question is empirical and easy to get wrong. On queries whose words overlap the gold answer, a strong lexical baseline such as BM25 [5] is hard to beat, and any apparent routing win could simply reflect easy retrieval. We therefore stress-test the architecture where lexical matching becomes unreliable—under paraphrase and under *adversarial* queries that embed misleading terms. Figure 1 previews the headline: BM25 is excellent on keyword-aligned queries and brittle under adversarial shift, while MoC-RAG degrades far more gracefully.

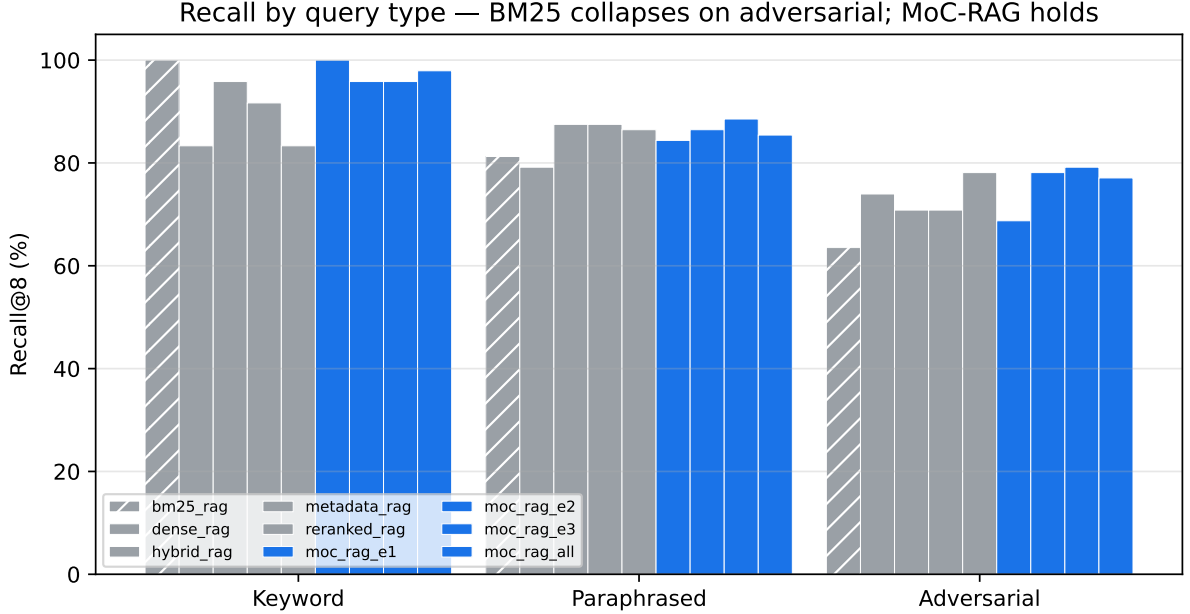


Figure 1: Recall@8 by query type (sentence-transformers embedder). BM25 leads on keyword-aligned queries but collapses under adversarial phrasing; MoC-RAG variants hold and overtake it on the adversarial split.

Contributions.

1. **MoC-RAG**, a routed retrieval architecture with a hybrid expert router (centroid + keyword + type + scope + activity priors) and an inspectable, budgeted pack assembler (§3).
2. A **public typed agent-memory benchmark** with five hard-negative kinds and five query styles, including parallel keyword / paraphrased / adversarial test splits that isolate robustness from topic leakage (§4).
3. An **evidence-based robustness result**: MoC-RAG preserves higher recall under adversarial lexical shift and roughly halves hard distractors versus the dense baseline family, at 95–100% routing accuracy (§6), together with a fully reproducible figure/table pipeline.

2 Background and Related Work

Sparse routing. The organizing principle behind MoC-RAG is sparse routing: expose many specialists but activate only the few a given input requires. This is the mechanism by which mixture-of-experts models scale capacity without proportional compute [6]. Matrix Context applies the same principle not to model weights but to *context*, treating the memory store as a set of typed experts selected per query.

Retrieval and fusion. Classic dense RAG [3] retrieves nearest neighbours from one index; lexical retrieval via BM25 [5] remains a strong, hard-to-beat baseline when query and target share vocabulary. Hybrid systems combine the two; we fuse the lexical and dense rankings with Reciprocal Rank Fusion [2], which needs no score calibration between channels. Dense embeddings are computed with sentence-transformers [4]. Redundancy in the assembled pack is controlled with a maximal-marginal-relevance penalty [1].

MoC-RAG: route typed context experts before retrieving

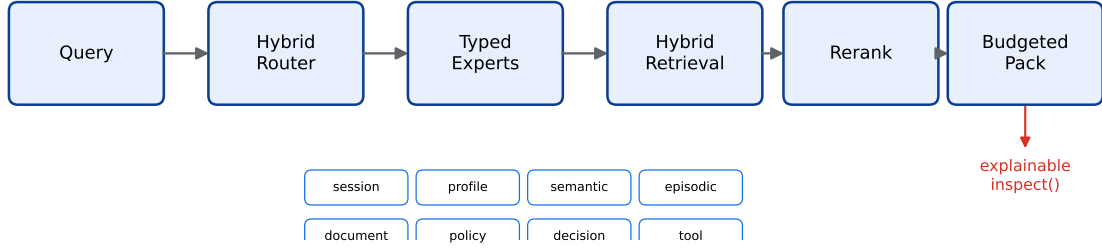


Figure 2: MoC-RAG routes a query to typed context experts before retrieving, then reranks and packs under a token budget; every choice is explainable.

Is this just metadata filtering? A natural objection is that routing to typed experts is equivalent to filtering a flat index by metadata. We treat this head-on by including a metadata-filtered hybrid RAG baseline that filters candidates by the query scope before retrieving (§4). MoC-RAG differs in two ways: it routes by *learned and prior-weighted typed relevance* rather than a hard metadata predicate, and it widens its expert selection under uncertainty rather than returning an empty or wrong-typed set. The benchmark measures whether this matters.

Agent memory. The contemporary agent-memory landscape—personalization, temporal knowledge graphs, operating-system-style memory management for long-running agents, and graph reasoning over private corpora—reflects the state of the field in early 2026. Matrix Context does not claim benchmark supremacy over these systems; its position is a narrower combination they do not bundle: a local-first default, typed and inspectable routing, and native fit within a single agent ecosystem.

3 Method: Matrix Context / MoC-RAG

MoC-RAG is a four-stage pipeline—route, retrieve, rerank, pack—over a store of typed context items, with every stage exposed for inspection (Figure 2).

Typed context items. The unit of memory carries content, an *expert* partition, a *type*, a hierarchical scope, an author-set importance, a confidence, an optional time-to-live, and an embedding. Typing the item is what enables routing by meaning and, later, governance by policy.

Hybrid expert router. Given a query q the router scores each expert e as a weighted sum of a dense centroid similarity and four cheap priors:

$$\text{score}(e) = 0.40 \cos(q, c_e) + 0.25 \text{kw}(q, e) + 0.15 \text{type}(q, e) + 0.10 \text{scope}(q, e) + 0.10 \text{act}(e), \quad (1)$$

where c_e is the expert centroid (mean item embedding blended with a seed description), kw is keyword-prior overlap, type a query-intent type prior, scope a scope-membership prior, and act a recent-activity prior. The top- k experts are selected; on low confidence or an indecisive margin the router *widens* selection rather than guessing—graceful degradation that is exactly the seam where a learned classifier later plugs in.

Table 1: MoC-RAG Benchmark statistics (deterministically generated).

Statistic	Value
Context items	1000
Queries	600
Context experts	8
Context types	10
Domains	6
Query variants	5
Hard-negative kinds	5
Train / Val / Test queries	360 / 120 / 120
Adversarial test queries	48

Hybrid retrieval. Within the selected experts, candidates are ranked by Reciprocal Rank Fusion [2] of a BM25 channel and a dense channel, so a weak channel cannot dominate.

Reranking. An optional local reranker re-scores the fused candidates by a normalized blend of dense, lexical, importance, confidence, and recency, leaving a cross-encoder seam for stronger rerankers.

Budgeted pack assembly. The pack is filled greedily under a token budget by $s(i) = w_R \text{rel}(i) + w_I \text{imp}(i) + w_T \text{rec}(i) - w_D \text{red}(i)$, where the redundancy term is a maximal-marginal-relevance penalty [1] that deduplicates near-identical memories at read time. Every kept and dropped item, its score breakdown, and the routing reason are returned by `inspect`, so the resulting context is auditable rather than a black box.

4 Benchmark Design

A small feasibility fixture is enough to show the mechanism but too keyword-aligned to settle the obvious objection. We therefore build a larger, public benchmark of typed agent memory, designed so that lexical overlap can be dialed down deliberately (Table 1).

Typed items and domains. The store spans six agentic-memory domains—project architecture, user/persona, code context, policy rules, tool/session history, and document QA—mapped onto eight typed experts and ten context types. Items carry importance, confidence, scope, and timestamps.

Hard negatives. Each gold fact is surrounded by five kinds of hard negative, the cases where flat retrieval is tempted: *same keyword, wrong expert*; *true fact, wrong scope*; *outdated (superseeded) decision*; *contradictory memory*; and *stale session note*.

Query variants. Crucially, every gold fact is queried five ways while the gold label is held constant: `direct` (keyword-aligned), `paraphrased`, `underspecified`, `cross_expert`, and `adversarial`. The adversarial phrasing deliberately embeds a misleading term—the same value asserted by the contradictory negative—so a flat lexical or dense retriever is genuinely lured toward the wrong item.

Parallel robustness splits. Splitting is performed at the *topic* level, so all variants of a topic stay in one bucket. The test topics are then phrased three ways into parallel `test_keyword`, `test_paraphrased`, and `test_adversarial` splits. Because the three splits cover the same topics, a change in score is attributable to query *style* rather than to topic leakage—this is what licenses the robustness comparison in §6.

Baselines. We compare BM25, dense, hybrid (BM25+dense RRF), metadata-filtered hybrid, and reranked RAG against MoC-RAG with `top_experts` $\in \{1, 2, 3, \text{all}\}$. All methods share one budgeted packer, so differences reflect retrieval and routing quality, not packer tricks.

5 Experiments

Setup. We evaluate with two embedders: an offline zero-dependency hashing embedder (a continuous-integration guard that requires no model download) and `sentence-transformers/all-MiniLM-L6-v2` as the competent embedder. Unless stated otherwise, results use the real embedder, a per-query token budget of 200, and $K = 8$ for ranking metrics. Each method indexes the same 1,000-item store and is evaluated on the held-out test queries.

Metrics. We report retrieval quality ($\text{Recall@}K$, $\text{Precision@}K$, MRR, $\text{nDCG@}K$), context efficiency (packed *hard distractors*, total tokens, useful-context ratio, and $\text{Recall@}K$ per 1k tokens), expert routing accuracy for routed methods, and latency. *Hard distractors* count items from a query’s labeled hard-negative set that enter the assembled pack—the harmful context a good router should keep out.

Reproducibility. The benchmark is deterministic (seeded) and every number, figure, and table in this paper is regenerated from committed result artifacts:

```
python -m benchmarks.moc_rag_benchmark.run build
python -m benchmarks.moc_rag_benchmark.run run      --split test --embedder st --groundedness
python -m benchmarks.moc_rag_benchmark.run compare --embedder st --groundedness
make figures    # docs/paper/latex/scripts/make_paper_figures.py -> figures/*.{pdf,svg}
make tables    # docs/paper/latex/scripts/make_paper_tables.py -> tables/*.tex
make paper     # compiles main.tex -> build/matrix_context_moc_rag.pdf
```

The dataset is published at huggingface.co/datasets/ruslanmv/moc-rag-benchmark and the software (Apache-2.0) at github.com/agent-matrix/matrix-context.

6 Results

Aggregate test split. On the mixed test split, which blends all five query styles, MoC-RAG with `top_experts=3` posts the best aggregate Recall@8 (86%) and the strongest MRR/nDCG, ahead of BM25 (78%) and the dense family (78–82%), while carrying roughly half their hard distractors at 95–100% routing accuracy (Table 2). We stress that this aggregate already reflects the harder query mix; on a purely keyword-aligned split BM25 is co-leading, as the robustness breakdown makes explicit.

Robustness by query type. Table 3 and Figures 1, 3 report the central result. BM25’s Recall@8 falls from 100% (keyword) to 64% (adversarial), a 36-point collapse, because misleading keywords pull it toward the contradictory negative. MoC-RAG holds within ~ 17 points (96% $\rightarrow$ 79% for $k=3$) and overtakes BM25 on the adversarial split by ~ 15 points. The dense, hybrid, metadata-filtered, and reranked baselines are less brittle than BM25 on recall but pay for it in harmful context.

Table 2: Main benchmark results on the mixed test split (real embedder). MoC-RAG rows shaded. HardDistr and Tokens: lower is better.

Method	Recall@8	Prec@8	MRR	nDCG@8	HardDistr↓	Tokens↓	UsefulRatio↑	RouteAcc
bm25_rag	78%	19%	0.719	0.727	167	23560	12%	—
dense_rag	78%	19%	0.558	0.617	271	23539	12%	—
hybrid_rag	82%	21%	0.723	0.740	244	23519	12%	—
metadata_rag	82%	20%	0.703	0.721	273	23364	13%	—
reranked_rag	82%	21%	0.717	0.745	249	23499	12%	—
moc_rag_e1	81%	20%	0.748	0.760	104	23509	12%	84%
moc_rag_e2	85%	21%	0.768	0.781	122	23547	13%	95%
moc_rag_e3	86%	22%	0.789	0.796	156	23529	13%	99%
moc_rag_all	85%	21%	0.766	0.779	196	23568	12%	100%

Table 3: Robustness by query type (Recall@8 and Hard Distractors per split; Δ is the keyword→adversarial recall change in points).

Method	R@8 key.	R@8 par.	R@8 adv.	HD key.	HD par.	HD adv.	Δ kw→adv
bm25_rag	100%	81%	64%	27	68	72	-36
dense_rag	83%	79%	74%	64	104	103	-9
hybrid_rag	96%	88%	71%	52	101	91	-25
metadata_rag	92%	88%	71%	63	107	103	-21
reranked_rag	83%	86%	78%	56	96	97	-5
moc_rag_e1	100%	84%	69%	22	40	42	-31
moc_rag_e2	96%	86%	78%	26	48	48	-18
moc_rag_e3	96%	89%	79%	30	64	62	-17
moc_rag_all	98%	85%	77%	34	82	80	-21

Harmful context. Figure 4 shows packed hard distractors on the adversarial split: MoC-RAG carries 48–62 versus 91–103 for the dense family—roughly half. Figure 5 places methods on a recall vs. hard-distractor Pareto plane; MoC-RAG variants dominate the upper-right (more recall, less harmful context).

Ablation over expert fan-out. Table 4 varies `top_experts`. Routing accuracy rises monotonically with k (Figure 6), reaching 100% at $k=3$ and `all`; $k=2$ and $k=3$ give the best recall/robustness trade-off, while $k=1$ minimizes hard distractors at some recall cost. Widening to all experts recovers a flat-RAG-like candidate pool and erodes the distractor advantage—evidence that the *selection*, not merely the typed store, drives the gains.

7 Discussion

The results support a careful, falsifiable claim rather than a sweeping one. **BM25 is not weak**—it is the best method on keyword-aligned queries, and a benchmark that hid this would not be credible. What changes under paraphrase and adversarial phrasing is that surface overlap stops being a reliable signal, and a retriever that ranks purely on words is led toward the very negatives the benchmark plants. MoC-RAG’s advantage is therefore best described as *robustness and context efficiency under typed, distractor-heavy conditions*: it preserves more recall when lexical matching degrades, and it keeps roughly half as much harmful context in the pack at equal token budget.

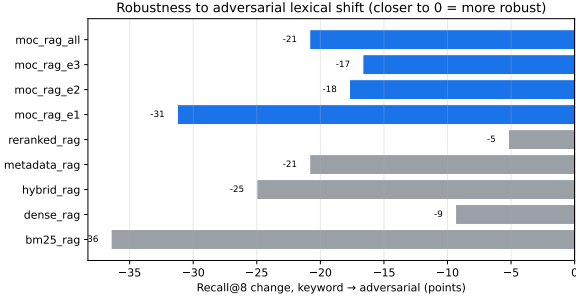


Figure 3: Recall change, keyword→adversarial (closer to 0 is more robust).

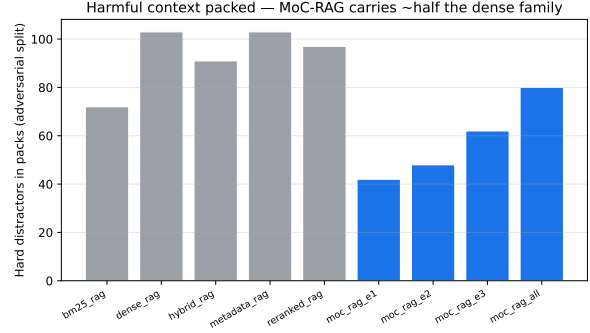


Figure 4: Hard distractors packed on the adversarial split (lower is better).

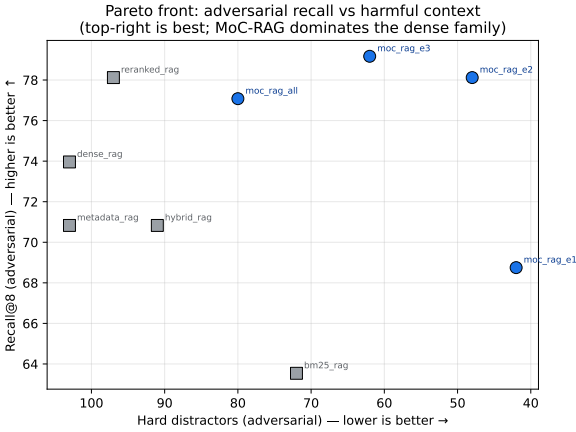


Figure 5: Pareto front: adversarial recall vs. hard distractors.

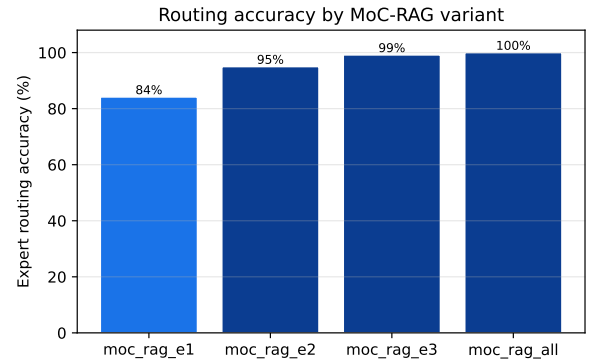


Figure 6: Expert routing accuracy by MoC-RAG variant.

Two pieces of evidence indicate *why*. First, routing accuracy tracks the robustness gain: variants that route correctly (95–100%) are exactly those that resist adversarial shift, and widening to all experts—which dilutes the routing decision—erodes the advantage. Second, the same qualitative flip appears with the offline hashing embedder, where the dense channel is weak: the hybrid router still routes well because its keyword, type, and scope priors do not depend on embedding quality. The gain is thus attributable to typed selection, not embedding strength alone.

Practically, this matches where agent memory actually lives. Real users paraphrase, omit specifics, and ask questions whose obvious keywords point at the wrong, outdated, or contradictory memory. In that regime, selecting the right typed experts before retrieving is a more reliable lever than ranking a single flat index, and the inspectability of the decision is a product feature in its own right.

8 Limitations and Reproducibility

Limitations. The benchmark is *synthetic* and English-only. Items and queries are template-generated, and the adversarial phrasings, while designed to be genuinely misleading, are derived from the same templates as the gold facts; a determined lexical baseline tuned to the templates could narrow the gap. The dataset is also modest in scale (1,000 items, 600 queries). We do not claim that MoC-RAG universally beats RAG: on keyword-aligned retrieval BM25 is at least as good, and on tasks without typed structure the routing layer adds cost without benefit. Finally,

Table 4: Ablation: MoC-RAG expert fan-out k across query types.

Variant	R@8 key.	R@8 par.	R@8 adv.	HD adv.	RouteAcc
$k=1$	100%	84%	69%	42	75%
$k=2$	96%	86%	78%	48	96%
$k=3$	96%	89%	79%	62	100%
all	98%	85%	77%	80	100%

the answer-quality column (grounded generation with a small local model) is wired but not run end to end here; the results above are retrieval- and context-level.

Threats to validity. Because splitting is by topic and the three variant splits are parallel, the robustness comparison is insulated from topic leakage; but it remains a comparison within one synthetic distribution. The hybrid router’s prior weights are fixed, not learned, and were not tuned on the test split.

Reproducibility. Generation is seeded and deterministic; baselines, metrics, figures, and tables are produced by committed scripts from committed result artifacts (§5). The dataset, the result JSONs, and a leaderboard space are public, and the software is Apache-2.0. The next step on the roadmap is to grow the dataset toward human-reviewed, naturally paraphrased, real long-horizon memory, and to promote the hybrid router into the core engine only once the gains replicate off the synthetic distribution.

9 Conclusion

We presented Mixture-of-Contexts RAG, a routed, inspectable retrieval architecture for agent memory that selects typed context experts before retrieving, and a public benchmark designed to test it where it matters: under paraphrased and adversarial queries with planted hard negatives. The evidence is specific and bounded. A strong lexical baseline collapses by 36 points from keyword to adversarial queries, while MoC-RAG holds within ~ 17 points, overtakes that baseline on the adversarial split, and packs roughly half the harmful context of the dense baseline family at 95–100% routing accuracy. The advantage is robustness and context efficiency under typed, distractor-heavy conditions—not universal supremacy over all RAG.

This reframes the contribution from “fewer distractors” to a testable robustness claim, backed by a reproducible dataset, harness, and figure pipeline. The artifacts—software at github.com/agent-matrix/matrix-context, dataset at huggingface.co/datasets/ruslanmv/moc-rag-benchmark, and a leaderboard space—let anyone reproduce, contest, or extend the result. We hope the benchmark, more than the method, is the lasting contribution: a place where the value of typed context routing can be measured honestly.

Availability

Software: Apache-2.0, github.com/agent-matrix/matrix-context. Dataset: [ruslanmv/moc-rag-benchmark](https://huggingface.co/datasets/ruslanmv/moc-rag-benchmark) (Hugging Face). A Zenodo DOI is minted on tagged release (see `.zenodo.json`, `RELEASE.md`).

References

- [1] Jaime Carbonell and Jade Goldstein. The use of MMR, diversity-based reranking for re-ordering documents and producing summaries. In *Proceedings of the 21st International ACM SIGIR Conference*, pages 335–336, 1998.
- [2] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Buettcher. Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference*, pages 758–759, 2009.
- [3] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [4] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2019.
- [5] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.
- [6] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarczyk, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations (ICLR)*, 2017.